

Project 1 – Highly Available Static Website on AWS using EC2, Application Load Balancer (ALB) & Auto Scaling.

prepared By: Kritika Agarwal

Project: Highly available Static website on AWS using EC2, ALB & Auto Scaling.

Objective

- Login AWS
- Project Purpose

Architecture Structure

- Structure
- Explanation

AWS Services Used

- Amazon EC2
- Application Load Balancer (ALB)
- Auto Scaling Group (ASG)
- Amazon Machine Image (AMI)
- EBS Snapshot
- Security Groups
- Apache Web Server (httpd)
- Amazon Linux

Testing Outcome

- Created
- Uploaded

Conclusion

- Summary
- Learning

1. OBJECTIVE-

- Login to AWS Console.
- Search EC2.
- Open the EC2 Dashboard.

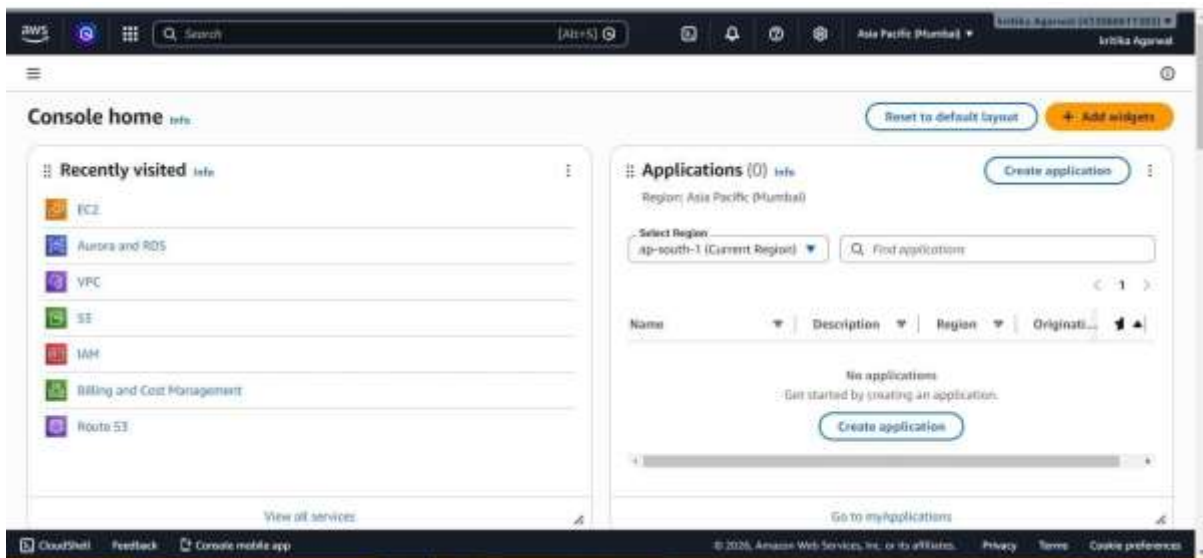


FIG-1: AWS CONSOLE HOME PAGE

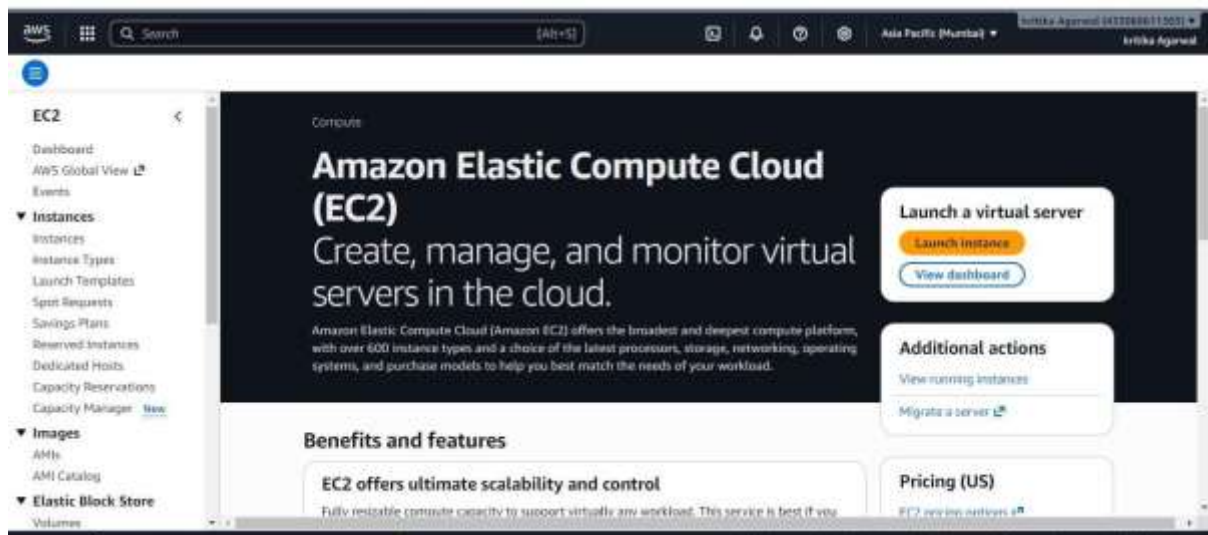


FIG-2: EC2 DASHBOARD

2. ARCHITECTURE STRUCTURE:

- User Browser: User browser se website access karta hai.
- ALB (Application Load Balancer): ALB user ki request ko available EC2 instance par bhejta hai.
- ASG (Auto Scaling Group): ASG traffic ke hisaab se EC2 instances ko automatically increase ya decrease karta hai.
- EC2 Instances (Apache Web Server): EC2 instance Apache Web Server ke through website ko host aur process karta hai.
- Static Website: Static Website user ko HTML, CSS aur JavaScript files serve karti hai.

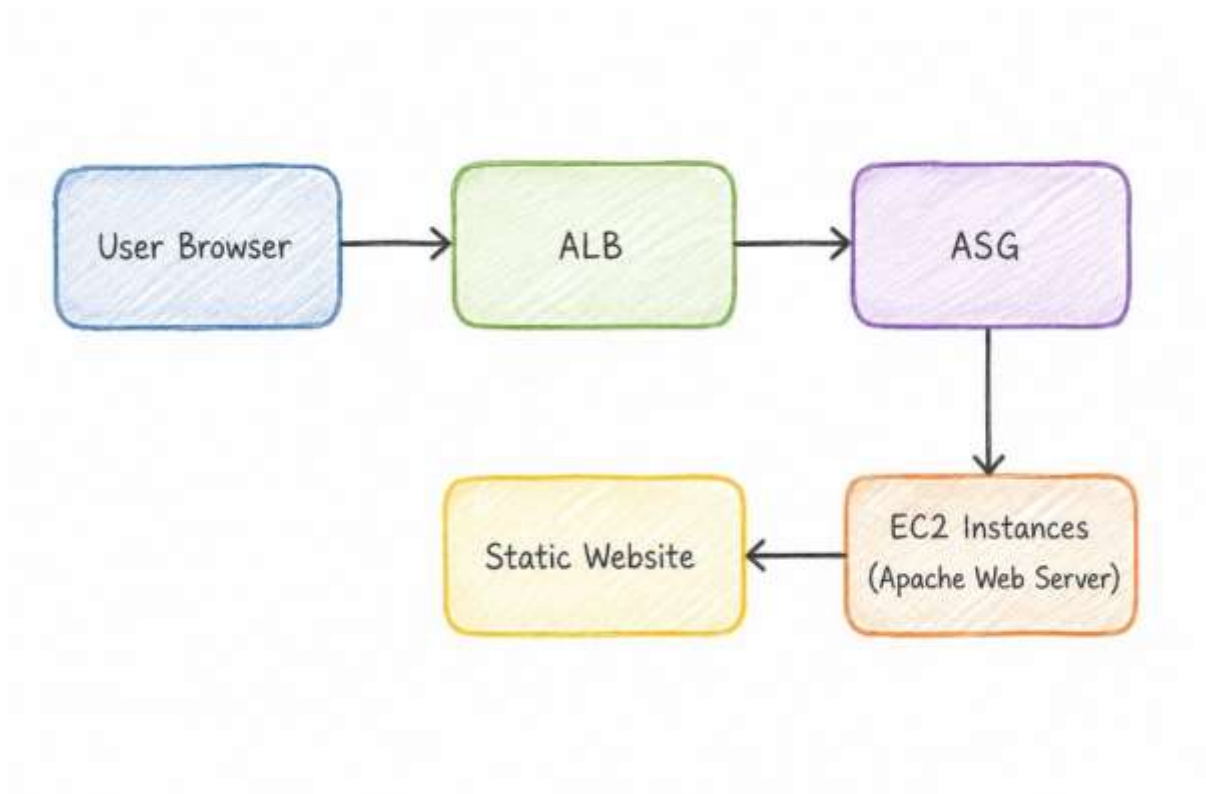


FIG: ARCHITECTURE DIAGRAM

3. AWS SERVICES USED-

Step 1: Launch an EC2 Instance:

1. Open the AWS Management Console.
2. Go to the EC2 Dashboard.
3. Click Launch Instance.
4. Select Amazon Linux AMI.
5. Choose t2.micro instance type.
6. Create or select an existing Key Pair.
7. Configure the Security Group:
 - SSH (Port 22)
 - HTTP(Port 80)
8. Launch the EC2 instance

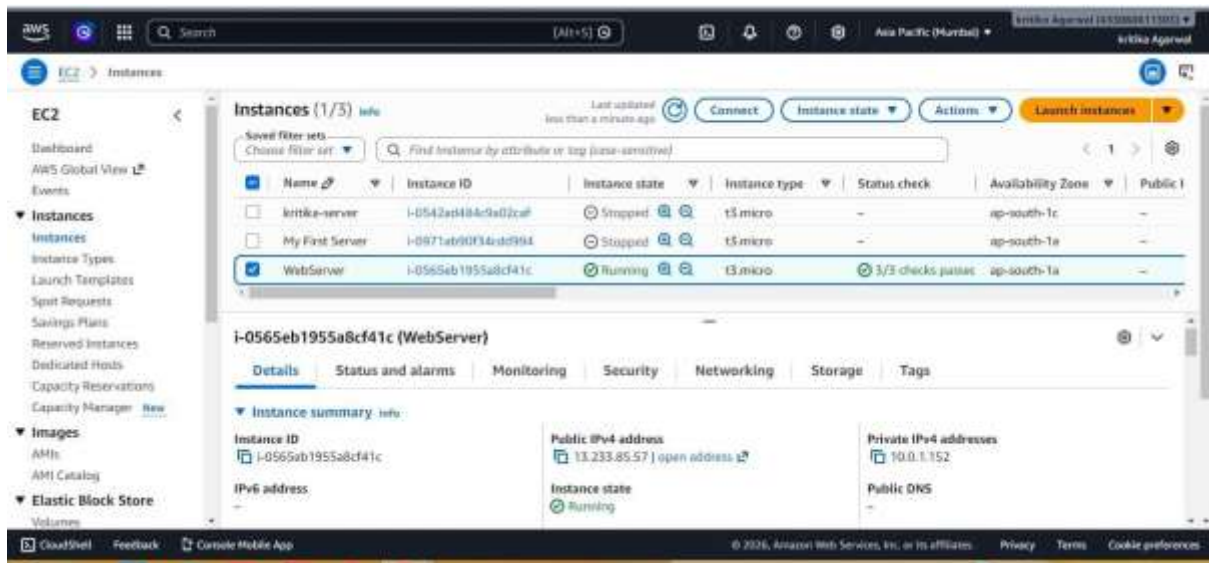


FIG-1: INSTANCE LAUNCH

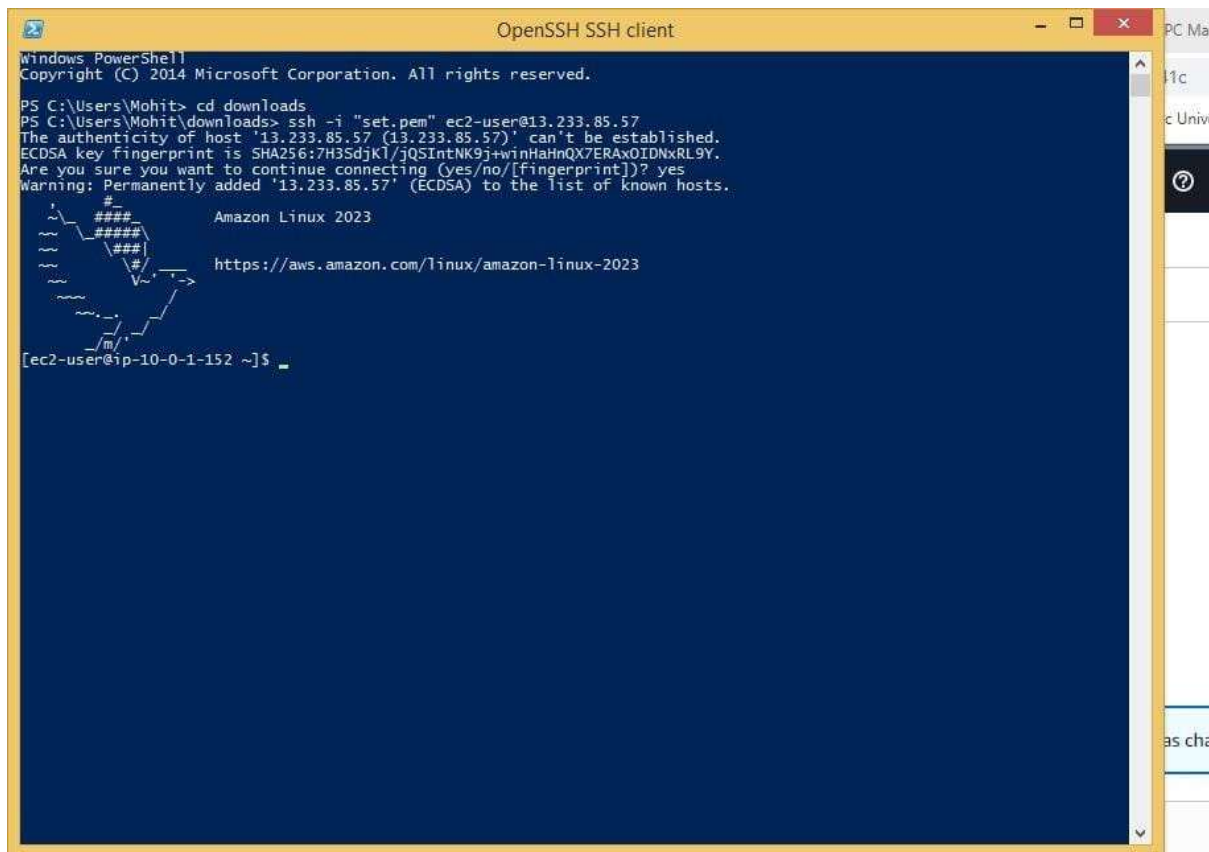


FIG-2: SSH CONNECTION

Step 2: Install Apache Web Server:

- ❖ Connect to the EC2 instance using SSH and run the following commands:
 - `sudo yum update -y`
 - `sudo yum install httpd -y`
 - `sudo systemctl start httpd`

- `sudo systemctl enable http`

```

[red@rp-10-0-140 ~]# yum yum install httpd -y
amazon linux 2023 kernel livepatch repository
Dependencies resolved.

Package                               Architecture      Version           Repository        Size
Installing:
httpd                                  x86_64            2.4.48-1.amzn2023.0.2  amazonlinux      46 k
Installing dependencies:
apr                                     x86_64            1.7.1-1.amzn2023.0.4  amazonlinux      120 k
apr-util                              x86_64            1.6.3-1.amzn2023.0.2  amazonlinux      97 k
apr-util-ldap                         x86_64            1.6.3-1.amzn2023.0.2  amazonlinux      13 k
apr-util-openssl                     x86_64            1.6.3-1.amzn2023.0.2  amazonlinux      16 k
apr-util-openssl-devel               x86_64            1.6.3-1.amzn2023.0.2  amazonlinux      1.4 M
httpd-filterstate                    x86_64            2.4.48-1.amzn2023.0.2  amazonlinux      12 k
httpd-tools                          x86_64            2.4.48-1.amzn2023.0.2  amazonlinux      90 k
libevent                             x86_64            2.0.9-4.amzn2023.0.2  amazonlinux      111 k
mailcap                              x86_64            2.1.49-1.amzn2023.0.2  amazonlinux      33 k
Installing weak dependencies:
apr-util-openssl                     x86_64            1.6.3-1.amzn2023.0.2  amazonlinux      16 k
mod_http2                            x86_64            2.0.42-1.amzn2023.0.2  amazonlinux      167 k
mod_lua                              x86_64            2.4.48-1.amzn2023.0.2  amazonlinux      59 k

Transaction Summary
Install 13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-1.7.1-1.amzn2023.0.4.x86_64.rpm                               1.6 MB/s | 128 kB 00:00
(2/13): apr-util-1.6.3-1.amzn2023.0.2.x86_64.rpm                          309 kB/s | 97 kB 00:00
(3/13): apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64.rpm                     845 kB/s | 13 kB 00:00
(4/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm                 794 kB/s | 16 kB 00:00
(5/13): httpd-2.4.48-1.amzn2023.0.2.x86_64.rpm                            1.3 MB/s | 46 kB 00:00
(6/13): httpd-core-2.4.48-1.amzn2023.0.2.x86_64.rpm                      41 MB/s | 1.4 MB 00:00
(7/13): httpd-filterstate-2.4.48-1.amzn2023.0.2.x86_64.rpm                41 MB/s | 12 kB 00:00
(8/13): httpd-tools-2.4.48-1.amzn2023.0.2.x86_64.rpm                     12.4 MB/s | 89 kB 00:00
(9/13): libevent-2.0.9-4.amzn2023.0.2.x86_64.rpm                         15 MB/s | 115 kB 00:00
(10/13): mailcap-2.1.49-1.amzn2023.0.2.x86_64.rpm                       1.1 MB/s | 33 kB 00:00
(11/13): mod_http2-2.0.42-1.amzn2023.0.2.x86_64.rpm                     5.9 MB/s | 167 kB 00:00
(12/13): mod_lua-2.4.48-1.amzn2023.0.2.x86_64.rpm                       2.8 MB/s | 59 kB 00:00
Total: 12 MB/s | 2.4 MB 00:00

Running transaction check

```

FIG-1: APACHE INSTALL

[illegible]

FIG-2: RUNNING APACHE

4. TESTING OUTCOME:

Step 3: Deploy the Static Website:

- Copy the website files to the /var/www/html/ directory.
- Restart the Apache service.
- Verify that the website opens using the EC2 Public IP.

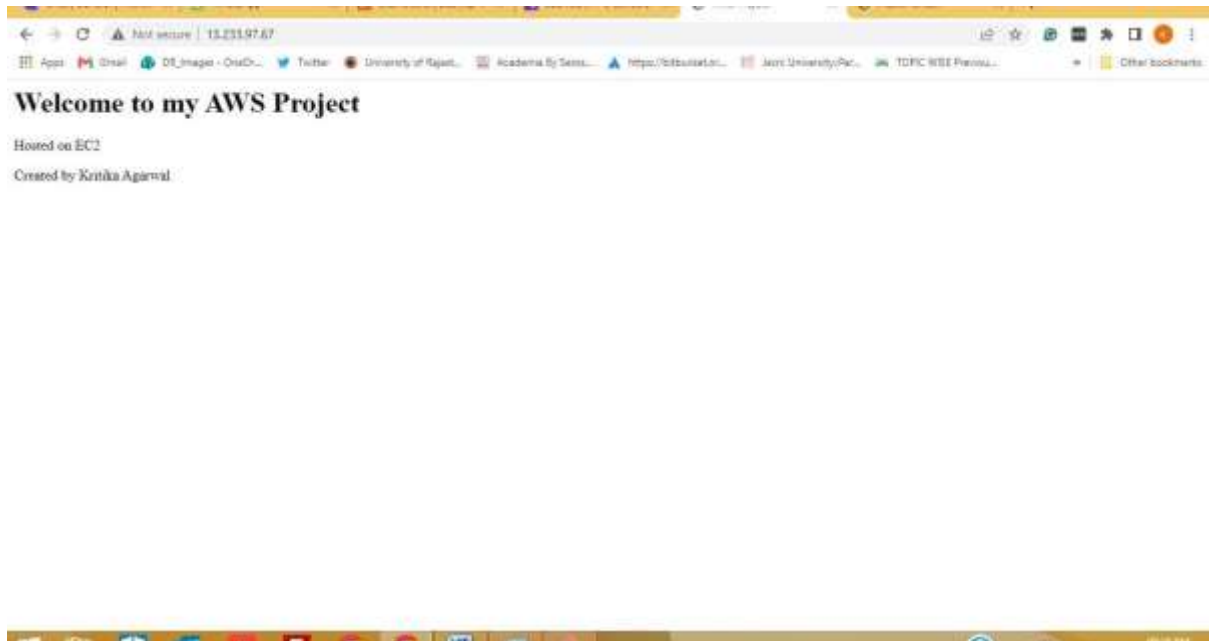


FIG: STATIC WEBSITE

Step 4: Create a Health Check Page:

- Create a file named health.html inside the /var/www/html/ directory.
- This page is used by the Application Load Balancer to verify that the web server is running correctly.

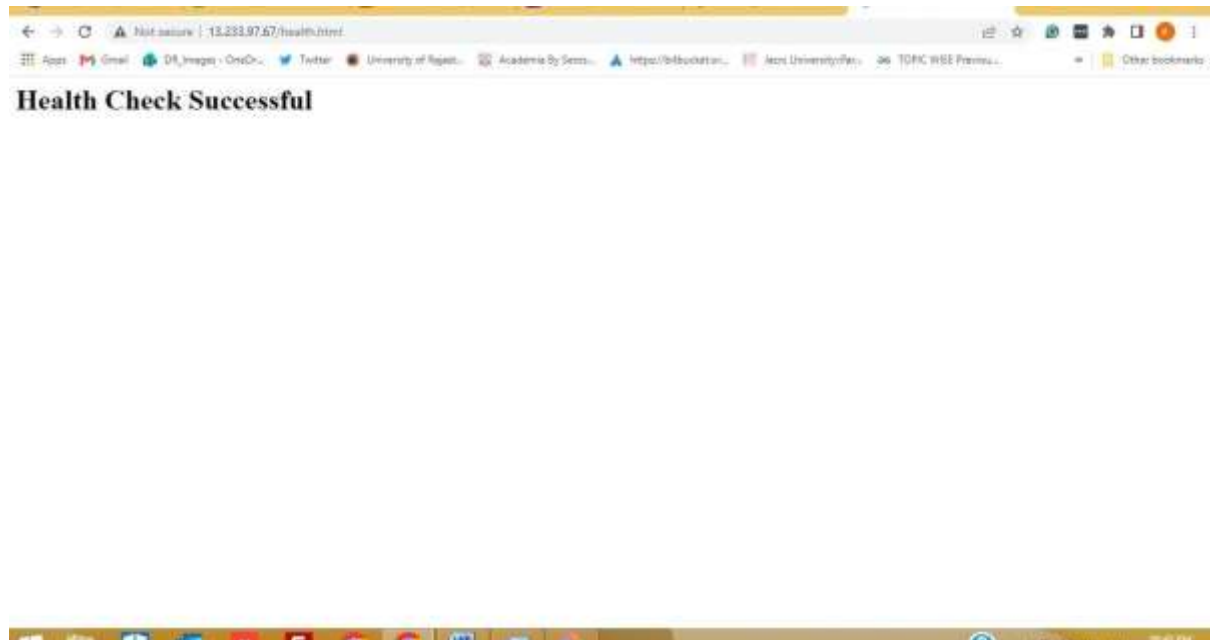


FIG: HEALTH STATUS

Step 5: Create a Golden AMI:

- Select the configured EC2 instance.
- Click Actions → Image and Templates → Create Image.
- Enter an image name.
- Create the AMI.

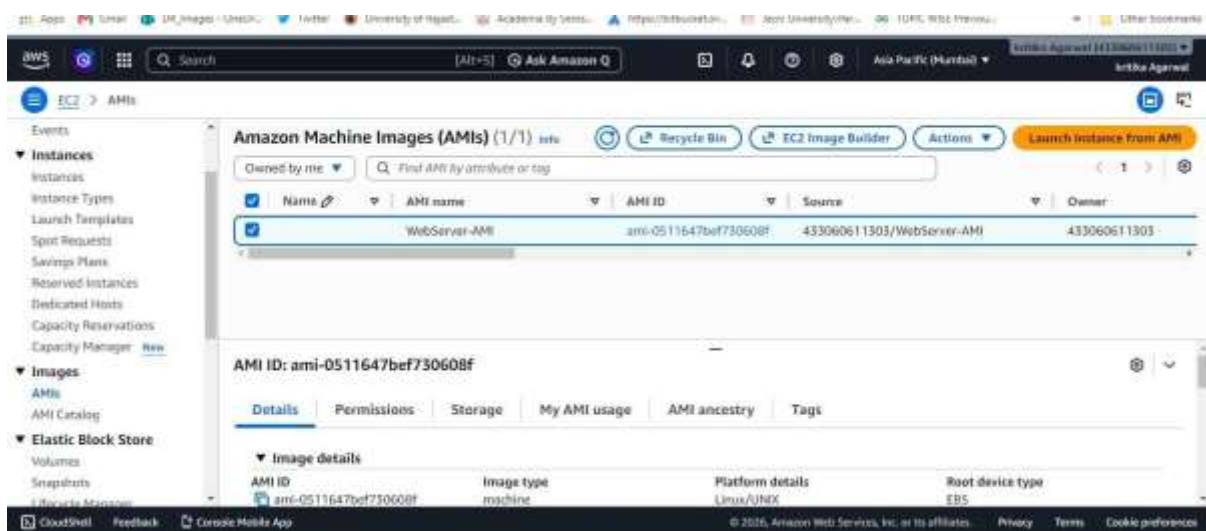


FIG: AMI

Step 6: Create a Target Group:

- Open the EC2 Dashboard.
- Select Target Groups.
- Click Create Target Group
- Select:
 - a. Target Type: Instances
 - b. Protocol: HTTP
 - c. Port: 80
- Configure the Health Check Path as:
/health.html
- Register the EC2 instance.

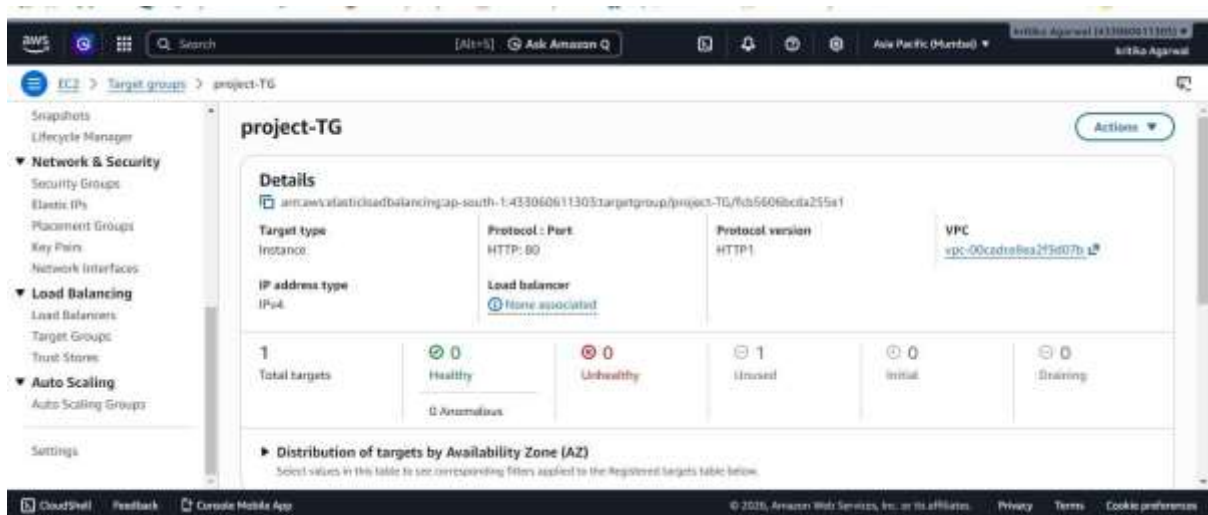


FIG: TARGET GROUP

Step 7: Create an Application Load Balancer:

- Open Load Balancers.
- Click Create Load Balancer.
- Select Application Load Balancer.
- Choose an Internet-facing Load Balancer.
- Select at least two public subnets.
- Attach the Security Group
- Attach the Target Group.
- Create the Load Balancer.

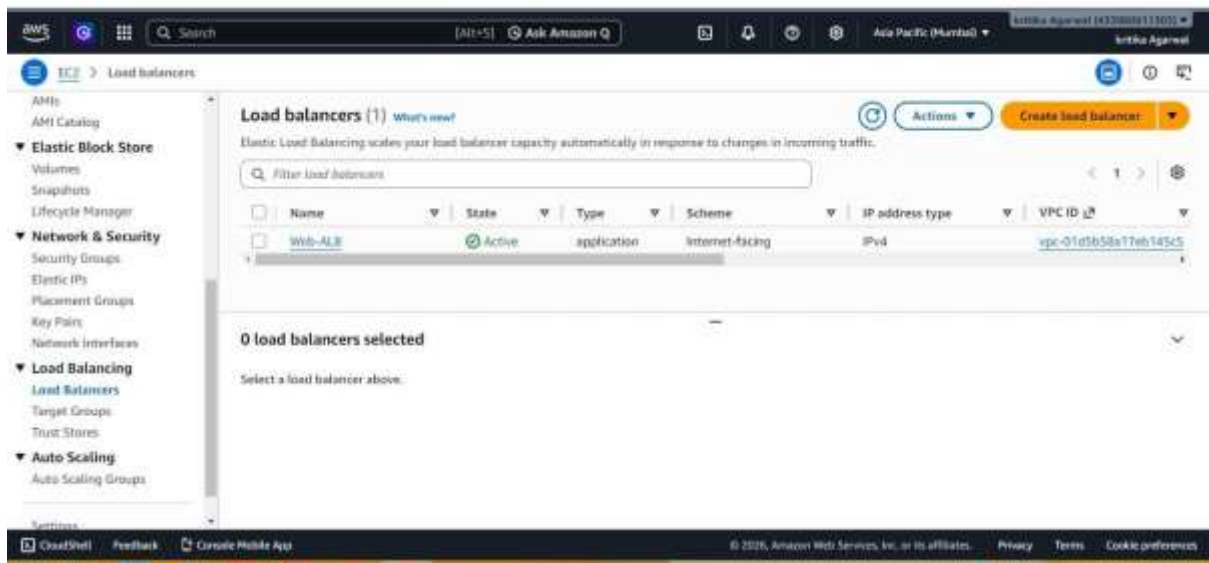


FIG: LOAD BALANCER

Step 8: Create an Auto Scaling Group:

1. Open Auto Scaling Groups.
2. Click Create Auto Scaling Group.
3. Select the Launch Template created from the Golden AMI.
4. Select the VPC and Public Subnets.
5. Attach the Target Group.
6. Configure:
 - Desired Capacity: 2
 - Minimum Capacity: 1
 - Maximum Capacity: 4
7. Create the Auto Scaling Group.

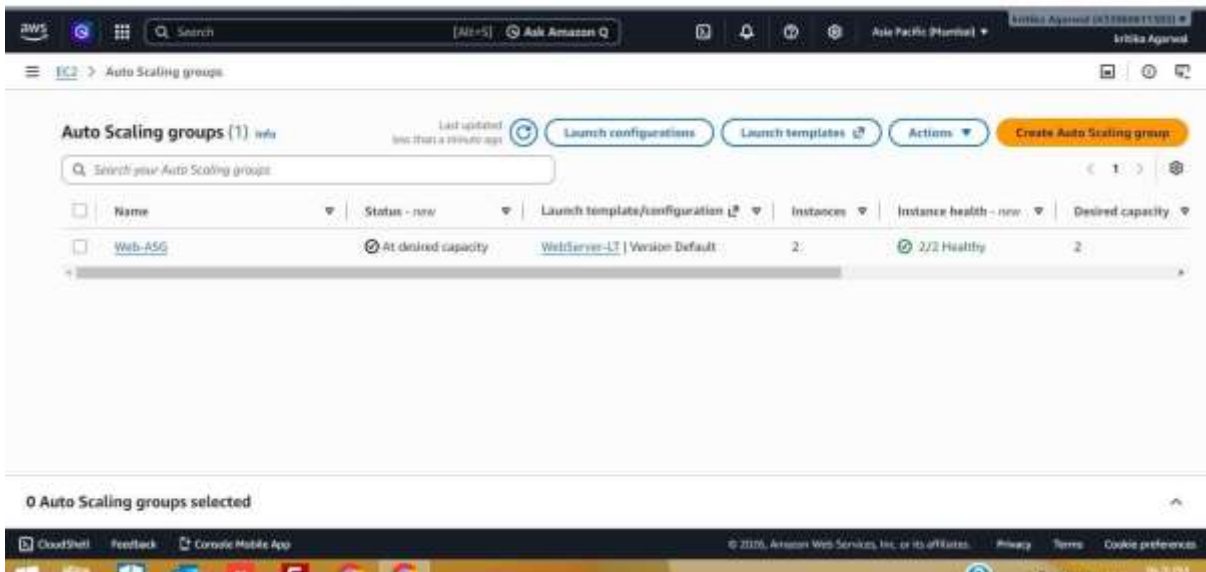


FIG: AUTO SCALING GROUP

Step 9: Test the Project:

- Open the Load Balancer DNS Name in a browser.
- Refresh the page several times to verify that requests are distributed across multiple EC2 instances.
- Stop or terminate one EC2 instance.
- Verify that Auto Scaling automatically launches a replacement instance.

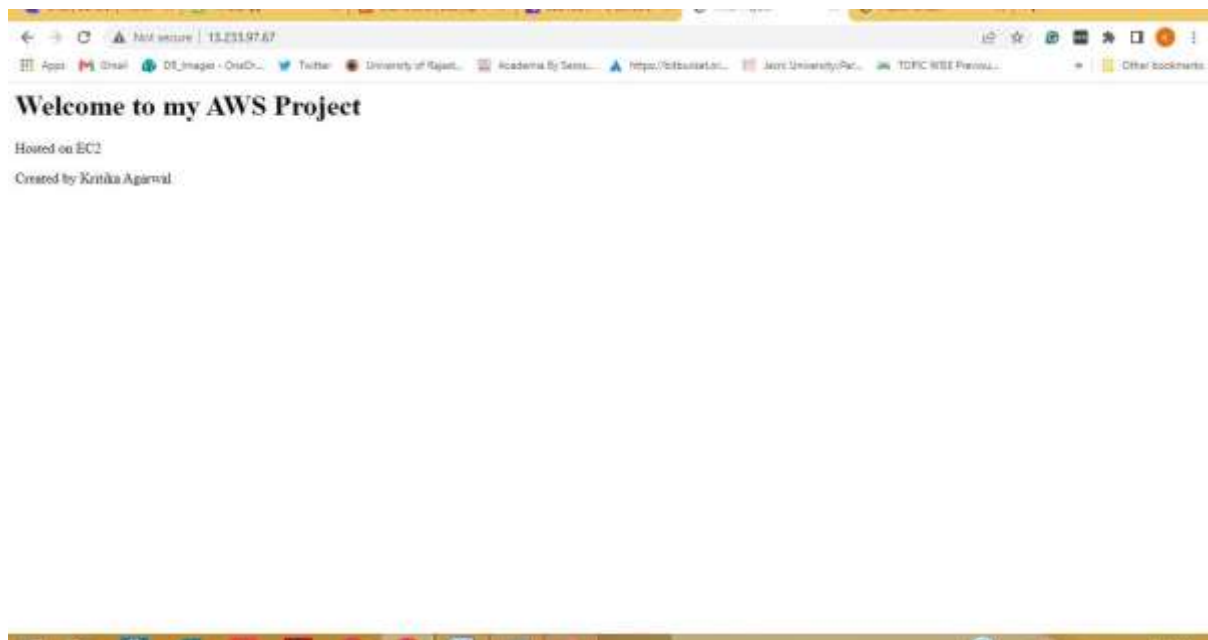


FIG: CONNECT SUCCESSFULLY

5. CONCLUSION:

SUMMARY-

- Static website is hosted successfully.
- Apache Web Server is running.
- Health check passes successfully.
- Application Load Balancer distributes traffic.
- Auto Scaling Group launches multiple EC2 instances.

LEARNING-

- How to launch and configure Amazon EC2.
- How to install and configure Apache Web Server.
- How to host a static website on EC2.
- How to create and use a Golden AMI.
- How to configure an Application Load Balancer.
- How to configure an Auto Scaling Group.

THANK YOU
